Dense Passage Retrieval and Reader Reranking on a 50K-Passage Wikipedia Corpus

Hồ Thành Tài, Lê Trần Bình An, Phan Bá Song Toàn
Trần Quang Long, Trịnh Minh Bảo Trân
Department of Artificial Intelligence, FPT University
Da Nang, 550000, Vietnam

***Abstract*—We study Dense Passage Retrieval (DPR) inference on a fixed reservoir sample of 50,000 Wikipedia passages. Full-test lexical, dense, and hybrid references cover five QA benchmarks and 28,219 questions. Using DPR's official regular-expression evaluator, CuratedTREC answer-match coverage is 87.75%, and coverage across the five tests is 65.68%. After passage-ID normalization, local BM25+DPR score fusion slightly exceeds DPR Multi at Hit@20 and Hit@100 on all five tests. We evaluate the official extractive Reader at $K = 100$ in FP16 after running TriviaQA development. Reader reranking raises CuratedTREC Hit@20 from 51.15% to 55.62% and TriviaQA Hit@20 from 33.60% to 41.35%. TriviaQA Reader span Exact Match is 12.54% on test. Per-question candidate IDs, relevance scores, and span predictions are retained. These are inference results on a reduced corpus and are not directly comparable to the original 21-million-passage benchmark.**

***Index Terms*—dense passage retrieval, passage reranking, corpus coverage, FAISS, open-domain question answering**

I. Introduction

Open-domain question answering depends on finding useful evidence in a large text collection. A retrieval system may identify a passage that discusses the topic of a question without placing a passage containing its answer near the top. This distinction becomes important when a lightweight first-stage retriever is followed by a more expensive neural reader. A system can lose answer-bearing evidence because the corpus omits it, because retrieval misses it, or because the final ranking places it below the evaluation cutoff.

Dense Passage Retrieval (DPR) provides a dual-encoder implementation and public model and data resources for studying this pipeline [1], [2]. Its original evaluation searches a Wikipedia collection of approximately 21 million passages. Reusing this full setting involves substantial storage, passage encoding, and indexing costs. A smaller collection makes an experimental pipeline more accessible, but also changes the task: it removes many possible answer passages along with many distractors. Results from a reduced collection therefore cannot be read as scores obtained under the original paper's conditions.

Our study fixes a reservoir sample of 50,000 passages and investigates how existing components behave under that constraint. We use exact inner-product search and retain the pretrained DPR weights. We then add a second stage that scores each retrieved question–passage pair with the relevance head of the official extractive DPR Reader. This setup permits a practical investigation of ranking improvements without attributing pretrained model training to the current experiments.

Three research questions guide the analysis:

1) How do corpus coverage and full-test answer-hit scores change when CuratedTREC answers are matched with the official DPR regular-expression evaluator?
2) How do the corrected BM25, DPR, and score-fusion baselines compare on the same 50,000-passage corpus?
3) What answer-hit and end-to-end span Exact Match results does the official Reader produce on the required TriviaQA evaluation at the requested $K = 100$ budget after first running TriviaQA development?

The contribution is an empirical feedback revision with explicit scope. First, we apply DPR's required CuratedTREC regex matching consistently to corpus coverage, BM25, DPR, and Reader evaluation. Second, we repair passage-ID normalization in the BM25+DPR fusion and report the corrected full-test baseline with per-query scores. Third, we evaluate the Reader at the requested $K = 100$ setting, run TriviaQA development before the test, and report span Exact Match from the Reader's answer head. The released code preserves question and candidate identifiers, retriever ranks, Reader scores, and span predictions for audit.

The work does not introduce a new neural architecture, a new loss, or a new indexing algorithm. Its value is in the measured behavior, transparent implementation choices, and the separation of corpus and ranking limitations. These distinctions are necessary when presenting a small-corpus reproduction as scientific evidence rather than treating it as a direct substitute for the original benchmark.

II. Related Work

A. Lexical and Dense Retrieval

BM25 ranks passages from lexical evidence with term-frequency saturation and document-length normalization [3]. It remains a useful reference because lexical overlap can identify specific entities and expressions that a dense model may rank poorly. Our lexical reference uses Apache Lucene, with implementation details stated separately from those of the DPR paper.

DPR encodes a question and a passage independently with BERT-based encoders [1], [4]. Passage representations can be computed once and reused across questions. The independence

of the two encoders enables efficient vector search, while a fixed passage vector must support many potential questions. In our experiments the released weights are held fixed; the original contrastive training objective explains the model's provenance rather than describing a training run performed here.

B. Vector Search

FAISS provides methods for exact and approximate similarity search [5]. HNSW organizes vectors in a graph to obtain approximate nearest neighbors [6]. The DPR paper reports an HNSW deployment for its large corpus. At 50,000 passages, exact inner-product search is feasible and avoids approximate-search errors relative to the DPR scoring function. Exactness of vector search is distinct from answer relevance: a correctly retrieved highest-scoring vector can still correspond to an unhelpful passage.

C. Neural Reranking and Readers

BERT-based reranking is an established way to score a query and a passage jointly [4], [7]. Sequence-to-sequence relevance scoring is explored by monoT5 [8]; token-level late interaction is used by ColBERT and its later ColBERTv2 version [9], [10]; and RankGPT studies listwise LLM reranking [11]. We cite these as related methods, not as baselines evaluated in this study. The original DPR system also provides an extractive reader with passage-selection and answer-span heads [2]. We reuse its passage-selection head as a reranker; we do not use an independently trained MS MARCO reranker and do not optimize a new reader. This distinction matters because our answer-hit metric measures answer-containing passages, while the Reader span exact-match metric is reported separately where available.

Natural Questions and TriviaQA supply complementary question-answering benchmarks [12], [13]. We use the official DPR-formatted test resources, including the CuratedTREC, WebQuestions, and SQuAD resources distributed with that repository, rather than creating a random train–validation–test split of Wikipedia passages.

III. Methodology

A. Corpus Acquisition and Preprocessing

We sample the official DPR Wikipedia passage resource (`data.wikipedia_split.psgs_w100`). The recorded archive size is 4,694,541,059 bytes, and the manifest records 21,015,324 source passages. A streaming reservoir sampler selects 50,000 distinct passage IDs without replacement using seed 12345. Each source passage has the same inclusion probability. Sampling does not use test questions, answer strings, or known positive passages to construct an answer-enriched collection.

The sample is approximately 0.238% of the source collection. Its tab-separated records retain `id`, `text`, and `title`. Multiple passages may share an article title. No new passage segmentation is performed after sampling, and the official compressed source is retained separately. The corpus file used in the Kaggle reranking inputs and the local coverage measurement has the same SHA256 digest; the digest is recorded in the accompanying artifact manifest.

B. DPR Dual Encoder and Exact Search

Let q denote a question and p a passage. The DPR question and passage encoders produce independent 768-dimensional representations:

$$u = E_Q(q), \qquad v_p = E_P(p), \qquad s_D(q,p) = u^\top v_p. \quad (1)$$

The encoders use pretrained `bert-base-uncased` architectures and released DPR checkpoint weights. The retriever sequence limit is 256 tokens. Passage inputs use title and body text through the DPR tensorizer. Vectors are not unit-normalized for cosine similarity. Passage vectors are computed with the passage encoder corresponding to the selected retriever checkpoint and are associated with their passage IDs.

The original DPR training objective ranks a positive passage against explicit and in-batch negatives. For context, its per-question negative log-likelihood has the form

$$\mathcal{L}_i = -\log \frac{\exp s_D(q_i, p_i^+)}{\exp s_D(q_i, p_i^+) + \sum_{p^- \in \mathcal{N}_i} \exp s_D(q_i, p^-)}. \quad (2)$$

We do not recompute this loss or update encoder weights in the experiments reported here. In particular, test inference does not have a training batch size or training epoch count.

FAISS `IndexFlatIP` exhaustively scores the sampled passage vectors. The raw float32 matrix contains $50{,}000 \times 768 \times 4 = 153{,}600{,}000$ bytes, approximately 146.5 MiB, excluding ID mappings, model memory, and duplicate buffers. This is manageable at the selected corpus size. The published large-corpus DPR deployment uses HNSW with 512 stored neighbors, construction depth 200, and search depth 128 [1]. Both index wrappers are available in the original code. We do not report a paired HNSW–FlatIP experiment, so no measured accuracy increase is assigned to the index substitution itself.

C. Candidate Construction

For a question q, exact DPR search produces a candidate set

$$\mathcal{C}_K(q) = \mathrm{TopK}_{p \in \mathcal{P}}\, s_D(q,p). \quad (3)$$

The feedback reranking run uses $K = 100$, the candidate budget explicitly requested by the instructor. Each returned passage ID retrieves the title and body from the fixed corpus. The Reader input is constructed as

$$\texttt{[CLS]}\ q\ \texttt{[SEP]}\ t_p\ \texttt{[SEP]}\ p. \quad (4)$$

The implementation appends body tokens to the question–title encoding, then truncates or pads the combined sequence to the Reader limit of 350 tokens. It does not unconditionally append another separator after the body. Token type IDs distinguish the question segment from title and passage; the attention mask excludes padding. Gold answer strings are not supplied to the Reader.

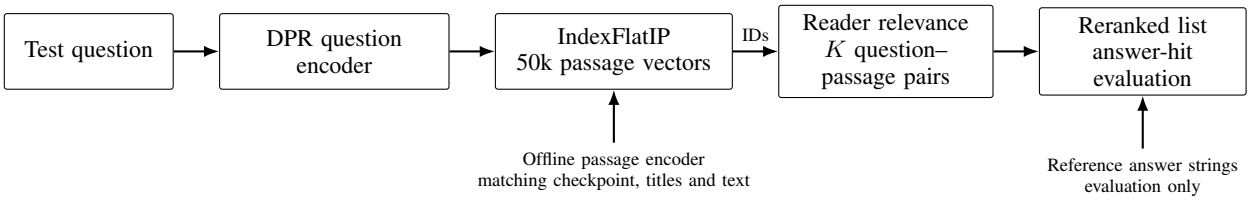


Fig. 1. Inference pipeline. Passage vectors are generated offline with the matching DPR checkpoint. Each Reader input contains one question and one candidate's title and text. Answers enter only the evaluation branch, after ranking.

D. Reader Relevance Reranking

Unlike the dual encoder, the Reader processes the concatenated question–passage input with one BERT encoder. Self-attention can connect tokens across the question, title, and passage. If $h_{\mathrm{CLS}}(q, t_p, p)$ is the final hidden vector of the initial token, the original `qa_classifier` computes

$$s_R(q,p) = w^\top h_{\mathrm{CLS}}(q, t_p, p) + b. \tag{5}$$

The reranked list sorts $\mathcal{C}_K(q)$ by s_R. We use raw relevance logits, not batchwise probabilities, and do not add the DPR score to the Reader score. For TriviaQA, we also decode a span from the candidate with the highest relevance score and report normalized Exact Match as a separate end-to-end metric. Answer-hit accuracy remains a retrieval diagnostic and is not replaced by Exact Match.

We use the official `nq-trivia-hybrid` Reader checkpoint; its full resource identifier is recorded in the artifact manifest. The registry describes it as a reader trained using Trivia multi-hybrid retrieval results [2]. This provenance motivates a possible dataset-alignment explanation, but does not establish how much of a measured gain is caused by training-domain alignment.

Reader evaluation uses `torch.inference_mode`. A batch contains 16 candidate pairs, represented as $[16, 1, 350]$ before the Reader flattens its first two axes. DataParallel divides the first axis across two T4 GPUs. Each pair is scored independently; there is no cross-passage attention between the 100 candidates. No parameters are updated, and the existing DPR passage vectors are reused.

E. Lexical Reference

The BM25 reference is a standalone Java runner using Lucene 8.11.2, `EnglishAnalyzer`, and `BM25Similarity` with $k_1 = 0.9$ and $b = 0.4$. It indexes article title and passage body together, while answer matching uses body text only. This is a Lucene reference, not the exact software deployment of the original paper.

F. BM25+DPR Score Fusion

The corrected hybrid baseline retrieves the top 2,000 candidates independently from Lucene BM25 and DPR, normalizes passage IDs to the common `wiki:<id>` form, and scores their union with

$$s_H(q,p) = s_{BM25}(q,p) + 1.1 s_D(q,p). \tag{6}$$

TABLE I
FULL TEST SIZES AND ANSWER-MATCH COVERAGE IN THE 50K CORPUS.

Test	Questions	Covered	Coverage (%)
NQ	3,610	2,192	60.72
TriviaQA	11,313	8,233	72.77
WebQuestions	2,032	1,388	68.31
CuratedTREC	694	609	87.75
SQuAD 1.1	10,570	6,112	57.82

The system reports the top 100 passages under this raw-score sum. The two score distributions are not normalized; the coefficient and candidate limits are recorded with the result. Per-query candidate IDs and component scores are retained for audit.

IV. Experimental Setup

A. Datasets and Evaluation Scope

The first-stage evaluations cover all 28,219 test questions from the five official DPR-formatted benchmarks. The feedback Reader reruns cover the full CuratedTREC and TriviaQA test sets. We do not subsample or repeatedly redraw test questions for this report. WebQuestions and SQuAD are first-stage references. Table V lists the denominators and corpus coverage.

The DPR Multi checkpoint is the fixed first stage for Reader reranking. An additional official NQ-Single checkpoint is reported for context; its non-NQ rows are transfer evaluations. The feedback reranking budget is fixed at $K = 100$, as explicitly requested. The official TriviaQA development split is run before test evaluation; no test results are used to choose settings. Reader relevance and answer-span outputs are computed in FP16 on two Kaggle Tesla T4 GPUs. Earlier FP32 Reader results are not used as feedback-run results.

B. Answer-Hit Metric

Let A_q contain the accepted answer strings of question q, and let $m(a,p)$ indicate that the normalized token sequence of answer a occurs contiguously in the body of passage p. For ranked list $R(q)$,

$$\mathrm{Hit}\ @k = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\exists p \in R_k(q),\ a \in A_q : m(a,p)]. \tag{7}$$

For NQ, TriviaQA, WebQuestions, and SQuAD, we use Unicode NFD normalization, DPR SimpleTokenizer, and case-

insensitive token-phrase matching. CuratedTREC answer entries are regular expressions in the official DPR data, so we use DPR's `regex_match` with its case-insensitive, Unicode, multiline flags for that dataset. Titles are excluded from answer matching. All displayed rates are recomputed from integer hit counts; percentage-point differences use unrounded rates.

This follows the answer-containing-passage retrieval convention. It is not an annotated relevance judgment or end-to-end QA Exact Match. For TriviaQA, we separately report normalized Exact Match from the Reader span selected from the highest-relevance candidate at the evaluated budget. A passage can contain the answer string without supporting the required relation; aliases and common short answers can also affect answer-hit scores.

C. Coverage and Coverage-Conditioned Accuracy

Corpus coverage is measured over all 50,000 passage bodies with the same dataset-specific matcher:

$$\mathrm{Cov} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\exists p \in \mathcal{P},\ a \in A_q : m(a,p)]\,. \tag{8}$$

The string-answer diagnostic scans normalized passage tokens with a multi-pattern token matcher; CuratedTREC scans with the official DPR regex matcher. Neither uses retrieval scores. Under this evaluator, $\mathrm{Hit}\,@k \le \mathrm{Cov}$ for any retriever searching this corpus. Coverage is therefore an answer availability ceiling, not a score achieved by a model.

For questions whose answer string appears somewhere in the corpus, we additionally derive

$$\mathrm{Hit}\,@k \mid \mathrm{covered} = \frac{\text{number of hits at } k}{\text{number of covered questions}}. \tag{9}$$

This diagnostic changes the denominator. It must be reported alongside full-test accuracy, not substituted for it when comparing with a benchmark paper. The original full test remains the evaluation set; no questions are removed from the primary results.

D. Runtime and Reproducibility

The feedback Reader reruns use two Kaggle Tesla T4 GPUs, a global candidate batch of 16, and FP16 inference. Retriever question batches are eight. Passage embeddings, corpus records, and QA files are fixed inputs; the reranking corpus is byte-identical to the coverage corpus. Runtime is the script's reranking-loop timer, including pair preparation, Reader scoring, and hit matching, but excluding queueing, environment setup, model loading, initial question encoding, and FAISS search. First-stage timing from another environment is not used to claim an end-to-end speed ratio.

Per-query outputs retain question IDs, candidate passage IDs, original retriever ranks, Reader relevance scores, answer-hit labels, and Reader span predictions. TriviaQA development is evaluated first at the fixed $K = 100$ budget; its full test split is reserved for final evaluation.

V. Results

A. Full-Test Lexical, Dense, and Hybrid References

Table II reports Hit@20 and Hit@100 over all 28,219 test questions. DPR Multi scores exceed Lucene BM25 at both cutoffs on all five benchmarks. The corrected BM25+DPR fusion is slightly higher than DPR Multi at both cutoffs on each test set. The fusion uses the raw weighted sum specified in Section III-F; these values are specific to that score scale, coefficient, and candidate union.

B. Reader Reranking and Span Exact Match

The official DPR Reader was run at $K = 100$ in FP16 on the full CuratedTREC and TriviaQA test sets. TriviaQA development was run first at the same budget. Table III reports answer-containing-passage Hit@k before and after Reader reranking; CuratedTREC uses DPR's official regex matching. Reader reranking increases Hit@20 from 51.15% to 55.62% on CuratedTREC and from 33.60% to 41.35% on TriviaQA. At Hit@100, CuratedTREC remains at 68.01%, as reranking changes the order of the same 100 candidates rather than adding candidates. The TriviaQA Hit@100 values are 48.04% for DPR Multi and 48.06% after Reader reranking.

On TriviaQA development, Reader span Exact Match is 12.50% (1,105/8,837); on the held-out test set it is 12.54% (1,419/11,313). Answer-containing-passage Hit@k and span Exact Match measure different outcomes: retrieving an answer-bearing passage does not guarantee that the Reader predicts the exact answer span. Per-query files retain all 100 candidate passage IDs, retriever ranks, Reader relevance scores, answer-hit labels, and span predictions for audit.

C. Corrected CuratedTREC Evaluation and Corpus Coverage

Using DPR's official regular-expression matching changes CuratedTREC corpus coverage from the previous literal-string count of 217 to 609 of 694 questions (87.75%). The earlier 31.27% coverage estimate and the interpretation based on it are withdrawn. Across all five tests, 18,534 of 28,219 questions have an answer match somewhere in the 50,000-passage corpus (65.68%).

On the full CuratedTREC test, corrected Hit@20/Hit@100 is 40.92%/55.48% for BM25, 43.52%/63.83% for the NQ-Single transfer checkpoint, 51.15%/68.01% for DPR Multi, 55.62%/68.01% after Reader reranking, and 52.16%/68.16% for BM25+DPR. Reader reranking therefore improves the top-20 answer-hit rate by 4.47 percentage points over DPR Multi, while the top-100 hit count stays fixed at 472/694 because the candidate set is unchanged. Among the 609 covered questions, DPR Multi retrieves a matching answer in its top 20 for 58.29%; BM25+DPR reaches 59.44%. At top 100, the corresponding conditional values are 77.50% and 77.67%. These conditional rates use only covered questions and are reported alongside, not instead of, full-test accuracy.

TABLE II
FULL-TEST FIRST-STAGE REFERENCES. CELLS REPORT HIT@20 / HIT@100 (%).

Test	Lucene BM25	DPR NQ-Single	DPR Multi	BM25+DPR
NQ	18.86 / 29.72	22.85 / 35.57	25.68 / 37.34	26.12 / 37.95
TriviaQA	23.52 / 35.91	24.97 / 40.65	33.60 / 48.04	34.62 / 48.86
WebQuestions	26.97 / 38.48	28.94 / 44.00	36.27 / 48.18	37.06 / 48.92
CuratedTREC	40.92 / 55.48	43.52 / 63.83	51.15 / 68.01	52.16 / 68.16
SQuAD 1.1	16.15 / 27.59	18.66 / 31.75	21.42 / 34.37	22.50 / 34.91

TABLE III
ANSWER-CONTAINING-PASSAGE RESULTS BEFORE AND AFTER READER RERANKING AT $K = 100$.

Test	Method	Hit@1	Hit@5	Hit@20	Hit@100
CuratedTREC	DPR Multi	16.43 (114)	34.58 (240)	51.15 (355)	68.01 (472)
	Reader, $K = 100$	25.79 (179)	40.63 (282)	55.62 (386)	68.01 (472)
TriviaQA	DPR Multi	9.81 (1,110)	20.77 (2,350)	33.60 (3,801)	48.04 (5,435)
	Reader, $K = 100$	18.09 (2,046)	30.57 (3,458)	41.35 (4,678)	48.06 (5,437)

TABLE IV
TRIVIAQA READER SPAN EXACT MATCH AT $K = 100$.

Test	Exact Match (%)	Count
TriviaQA	12.54	1,419/11,313

TABLE V
FULL TEST SIZES AND ANSWER-MATCH COVERAGE IN THE 50K CORPUS.

Test	Questions	Covered	Coverage (%)
NQ	3,610	2,192	60.72
TriviaQA	11,313	8,233	72.77
WebQuestions	2,032	1,388	68.31
CuratedTREC	694	609	87.75
SQuAD 1.1	10,570	6,112	57.82

VI. DISCUSSION

A. What the CuratedTREC Correction Changes

CuratedTREC uses regular-expression answer references in the official DPR data. Evaluating them as literal strings undercounted answer-bearing passages and made the corpus appear unusually sparse for this test set. With the official regex evaluator, 87.75% of CuratedTREC questions have a matching answer expression somewhere in the corpus, the highest coverage among the five tests. The earlier explanation that CuratedTREC had low absolute accuracy mainly because its answer passages were absent is therefore withdrawn.

The corrected first-stage results show that DPR Multi reaches Hit@20 of 51.15% on CuratedTREC and the local BM25+DPR fusion reaches 52.16%. Reader reranking reaches 55.62% at Hit@20, a 4.47-point increase over DPR Multi. Its Hit@100 remains 68.01%, equal to the DPR candidate set's coverage at that cutoff: the Reader reorders the 100 passages but cannot retrieve a passage outside them. Among covered questions, DPR Multi and BM25+DPR reach 58.29% and 59.44% at Hit@20, respectively. Coverage-conditioned values describe ranking after answer availability is accounted for; they do not replace full-test accuracy or establish semantic support.

B. Reader Interpretation

On TriviaQA, the Reader raises Hit@20 from 33.60% to 41.35%; Hit@100 is effectively unchanged (48.04% versus 48.06%). The answer-span head achieves 12.54% normalized Exact Match on test, while 48.06% of questions have an answer-matching passage among the 100 reranked candidates. This gap reflects the difference between locating an answer-bearing passage and predicting its exact answer span. The experiment uses the released Reader checkpoint and evaluates inference; it does not train a new Reader.

VII. LIMITATIONS

The evaluation uses one fixed reservoir sample of 50,000 passages from the 21-million-passage Wikipedia corpus. Sampling changes both answer availability and the distribution of distractors. Measured coverage is an answer-string or regex availability diagnostic, not a semantic relevance judgment; a matching expression can occur in a passage that does not support the answer.

All DPR and Reader weights are released author checkpoints. The experiments evaluate inference and do not reproduce DPR training or update model weights. The corrected BM25+DPR baseline uses a raw weighted score sum; its values depend on the recorded component scores and coefficient.

The study uses one corpus seed and one run per configuration, so it does not estimate variation across corpus samples or establish statistical significance. IndexFlatIP gives exact inner-product results for the supplied vectors, but this revision does not measure an HNSW–FlatIP speed/recall tradeoff.

The Reader is evaluated on a finite $K = 100$ candidate set produced by DPR and cannot recover answer-bearing passages outside it. Span predictions are affected by the 350-token input limit. Reader Exact Match is meaningful for literal answer

strings; CuratedTREC is evaluated with regex-based answer-hit matching rather than span Exact Match. These are retrieval and reranking measurements, not a complete open-domain QA evaluation.

VIII. Conclusion

The corrected evaluation changes the interpretation of CuratedTREC: DPR regex matching finds an answer match for 609 of 694 questions in the 50,000-passage corpus, rather than the 217 counted by the former literal-string check. The corrected BM25, DPR, Reader, and BM25+DPR results use the official CuratedTREC regex evaluator. Reader reranking improves Hit@20 by 4.47 percentage points on CuratedTREC and 7.75 points on TriviaQA relative to DPR Multi; the unchanged candidate set limits gains at Hit@100. TriviaQA Reader span Exact Match is 12.54% on test. The ID-normalized BM25+DPR fusion is slightly above DPR Multi at Hit@20 and Hit@100 on each benchmark under the recorded raw-score combination. These findings describe inference on a 50,000-passage sample with released checkpoints, not a reproduction of DPR training or its full Wikipedia-corpus evaluation.

Author Contributions

Hồ Thành Tài: (1) Methodology and evaluation protocol. (2) Software for DPR full-test evaluation, BM25+DPR fusion, and the official Reader reranking experiments in Versions 1 and 2. (3) Formal analysis of corpus coverage and five-benchmark retrieval results, including corrected CuratedTREC regex evaluation and TriviaQA Reader span Exact Match. (4) Writing—original draft and project coordination.

Lê Trần Bình An: (1) FAISS IndexFlatIP retrieval configuration used in the experiments. (2) Validation of exact-search and retrieval outputs across the corpus. (3) Writing—review and editing.

Phan Bá Song Toàn: (1) Co-development with Hồ Thành Tài of regex matching for CuratedTREC. (2) Validation of CuratedTREC coverage and BM25, DPR, and Reader Hit@k results across the two versions. (3) Writing—review and editing.

Trần Quang Long: (1) Software implementation of the Lucene BM25 pipeline. (2) Literature review, reference curation, and verification. (3) Writing—review and editing.

Trịnh Minh Bảo Trân: (1) Methodology and data curation for the fixed 50,000-passage reservoir sample, including corpus preparation and preprocessing. (2) Dataset and result consistency checks for the benchmark runs. (3) Review of result tables and figures, and writing—review and editing.

References

[1] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020, pp. 6769–6781. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/

[2] Facebook Research, "DPR: Official code and download resource registry," https://github.com/facebookresearch/DPR, accessed November 3, 2026.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *arXiv preprint arXiv:1702.08734*, 2017. [Online]. Available: https://arxiv.org/abs/1702.08734

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *arXiv preprint arXiv:1603.09320*, 2016. [Online]. Available: https://arxiv.org/abs/1603.09320

[7] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019. [Online]. Available: https://arxiv.org/abs/1901.04085

[8] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin, "Document ranking with a pretrained sequence-to-sequence model," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 708–718. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.63/

[9] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: https://doi.org/10.1145/3397271.3401075

[10] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," *arXiv preprint arXiv:2112.01488*, 2021. [Online]. Available: https://arxiv.org/abs/2112.01488

[11] W. Sun *et al.*, "Is ChatGPT good at search? investigating large language models as re-ranking agents," *arXiv preprint arXiv:2304.09542*, 2023. [Online]. Available: https://arxiv.org/abs/2304.09542

[12] T. Kwiatkowski *et al.*, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: https://aclanthology.org/Q19-1026/

[13] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in *Proceedings of ACL*, 2017, pp. 1601–1611. [Online]. Available: https://aclanthology.org/P17-1147/